

QECO: QoE 중심 계산 오프로딩

QECO: A QoE-Oriented Computation Offloading Algorithm Based on Deep Reinforcement Learning for Mobile Edge Computing

Iman Rahmaty, Hamed Shah-Mansouri, Ali Movaghar (2025)

동적 edge workload에서 각 mobile device의 장기 QoE를 학습하는 방법

한 task가 만드는 선택 충돌

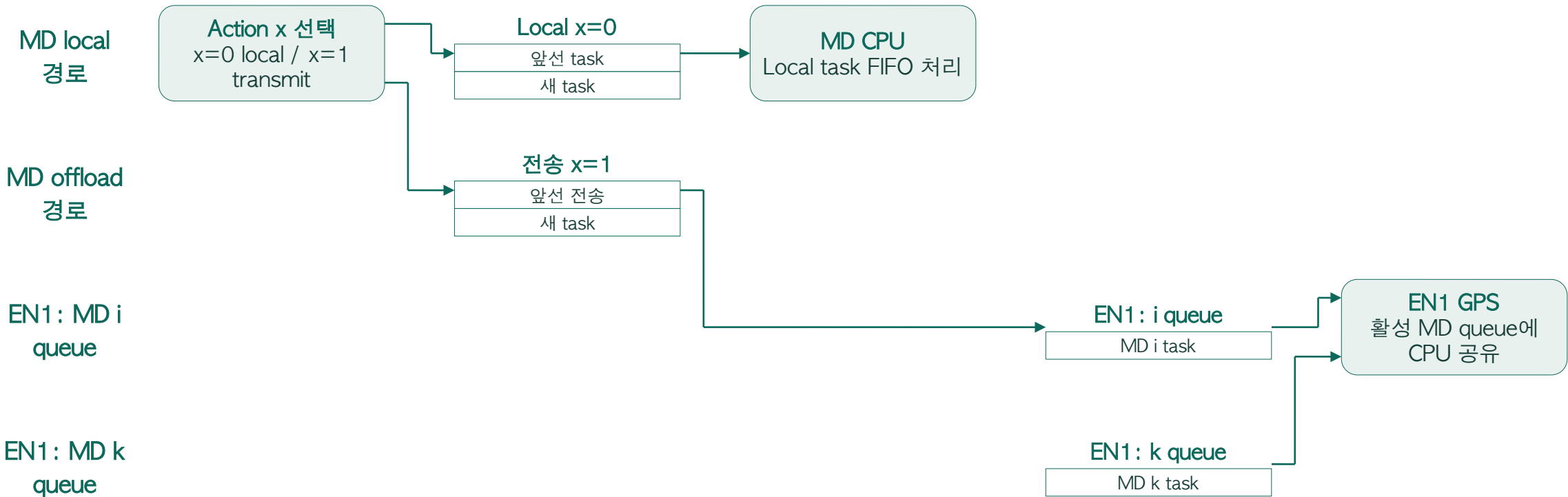
항상 local 또는 항상 offload하는 정책은 서로 다른 queue 병목을 만든다

선택	사용 자원	주요 위험	결정에 필요한 정보
Local 계산	MD CPU와 local queue	긴 대기과 deadline 초과	Local backlog, task 요구량
Edge offload	무선 전송과 EN CPU	전송 비용과 EN 혼잡	전송 queue, EN workload
QECO 선택	상태마다 route 변경	불확실한 미래 load	관측 상태와 workload history

오프로딩 문제는 빠른 server 선택보다 두 route의 queue 결과를 함께 예측하는 문제다

MD와 EN의 실제 queue 경로

$x=0$ 은 local FIFO, $x=1$ 은 transmission FIFO를 거쳐 선택 EN으로 가는 상호 배타적 action이다



세 queue의 역할

대기 위치와 처리 주체를 구분해야 state와 reward의 의미가 선명해진다

Queue	위치	처리 자원	논문 모델
Local-computation	각 MD	MD CPU	FIFO, deadline 시 처리 또는 drop
Transmission	각 MD	무선 link	FIFO, 선택 EN으로 전송
Edge-computation	각 EN의 MD별 queue	EN CPU	활성 queue끼리 GPS로 용량 공유
Active queue count	EN마다 $b_j(t)$	공유 자원 수요	Slot 끝에 EN이 broadcast

EN 하나가 모든 task를 한 줄로 처리하지 않고 MD별 queue에 용량을 나눠 준다

Deadline이 task 종료를 정하는 과정

Task는 route의 queue를 기다린 뒤 처리되며 deadline 전에 끝나지 않으면 drop된다

- | | | |
|----|------------|--|
| 01 | Task 도착 | Size와 CPU density, deadline을 가진 task가 slot t에 도착 |
| ↓ | | |
| 02 | Route 배정 | Local FIFO 또는 transmission FIFO와 선택 EN queue로 이동 |
| ↓ | | |
| 03 | Queue 대기 | 앞선 task와 EN의 active queue 수가 처리 시작을 늦춤 |
| ↓ | | |
| 04 | 처리 또는 drop | Deadline 안에 끝나면 완료, 끝내지 못하면 즉시 drop |

Queue 처리에 여러 slot이 걸리므로 action과 reward 관측 시점이 분리된다

연구 목표와 의사결정 단위

각 MD는 다른 MD의 task model이나 action을 받지 않고 자신의 장기 QoE를 최적화한다

- 의사결정 시점은 새 task가 도착한 time slot이다
- 정책은 관측 state를 local 또는 특정 EN action에 대응시킨다
- 목표는 discounted QoE의 기대 합을 각 MD별로 최대화하는 것이다
- 다른 MD의 action과 task model은 직접 입력하지 않는다
- EN workload history는 공유된 active queue count에서 얻는다

논문의 distributed 의미는 제한된 관측으로 각 MD가 독립 action을 고른다는 데 있다

MD가 보는 state snapshot

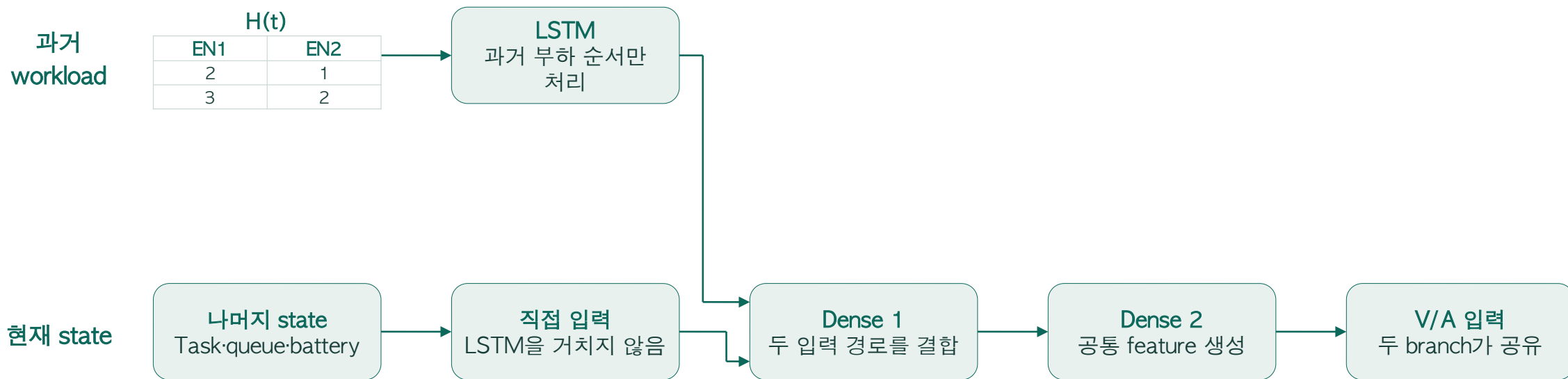
State는 새 task, MD queue, 직전 EN queue, battery mode, EN workload history를 묶는다

State 항목	관측 내용	결정에 주는 정보
$\lambda_i(t)$	새 task size	전송량과 계산량
δ_i^C, δ_i^T	Local과 transmission 대기 slot	MD 내부 backlog
$\eta_i^E(t-1)$	각 EN의 내 queue 길이	Offload target별 backlog
$\phi_i(t)$	Battery mode 또는 level	Delay와 energy 가중치
$H(t)$	T_s slot 동안 EN별 active queue 수	Workload 변화 패턴

$H(t)$ 는 미래 값을 주는 oracle이 아니라 과거 active queue count의 행렬이다

History만 LSTM을 통과한다

현재 state의 일반 항목과 EN workload sequence는 서로 다른 입력 경로를 거친다



Action은 route와 EN index를 함께 정한다

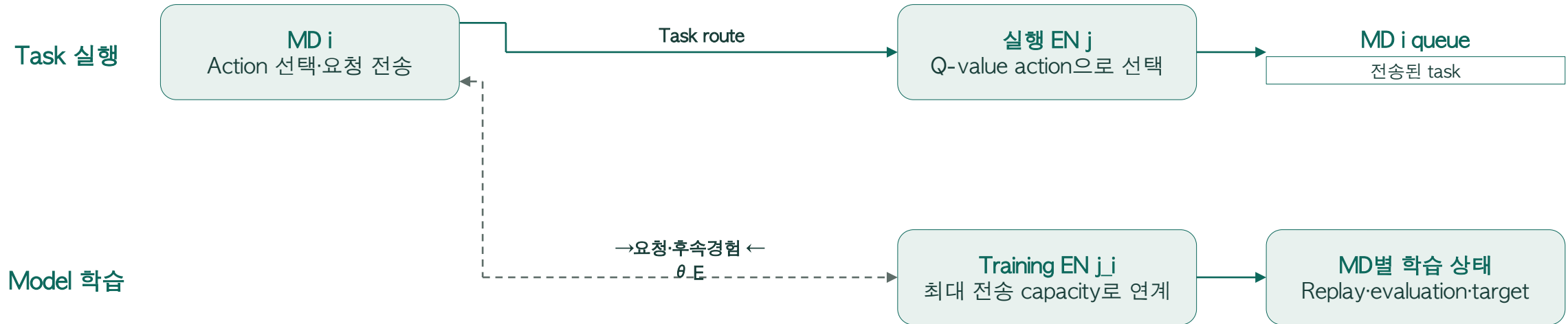
QECO는 CPU 주파수나 전송 power를 연속 제어하지 않고 discrete route를 선택한다

Action 구성	값	실제 효과	적용 조건
$x_i(t)$	0	Local queue에 배정	새 task 도착
$x_i(t)$	1	Transmission queue에 배정	새 task 도착
$y_{i,j}(t)$	선택 EN만 1	Offload target j 지정	$x_i(t)=1$
Action set	Local 1개와 EN별 offload	각 action의 Q-value 비교	ϵ -greedy 선택

한 offload action에는 전송 여부뿐 아니라 어느 EN queue로 보낼지도 포함된다

Offload target과 training EN의 구분

Solid는 task 실행, dashed는 방향이 다른 training-control message를 나타낸다



Task 실행 EN과 model 학습 EN은 선택 기준과 message가 다르다

QoE reward의 세 가지 근거

논문은 완료 여부를 reward 경계로 두고 delay와 energy를 battery weight로 결합한다

요소	논문 정의	Reward에 미치는 영향
Task 완료	Deadline 안에 processed	Completion reward R을 받음
Task delay D	도착부터 처리까지 slot 수	Cost의 ϕ D 항
Energy E	Local, 전송, edge, standby 비용	Cost의 $(1-\phi)$ E 항
Task drop	Deadline 전에 완료하지 못함	R 없이 -E만 받음

QoE는 표준 사용자 설문 점수가 아니라 이 MDP에서 task 결과를 평가하는 학습 reward다

Battery가 delay와 energy 비중을 바꾼다

$C = \varphi D + (1 - \varphi)E$ 이며 높은 φ 는 delay, 낮은 φ 는 energy의 비중을 키운다

Battery mode	φ 의 방향	Cost에서 강조	해석 경계
Performance	높은 쪽	Delay D	다른 state도 action에 영향
Power saving	중간	Delay와 energy 절충	고정 route를 뜻하지 않음
Ultra saving	낮은 쪽	Energy E	항상 offload를 뜻하지 않음
단위 경계	논문에 normalization 미기재	Slot과 energy를 한 cost로 결합	R의 수치도 Table II에 없음

Preference 구조는 분명하지만 물리 단위를 합친 범용 score로 옮길 근거는 부족하다

정규화된 교실 예시

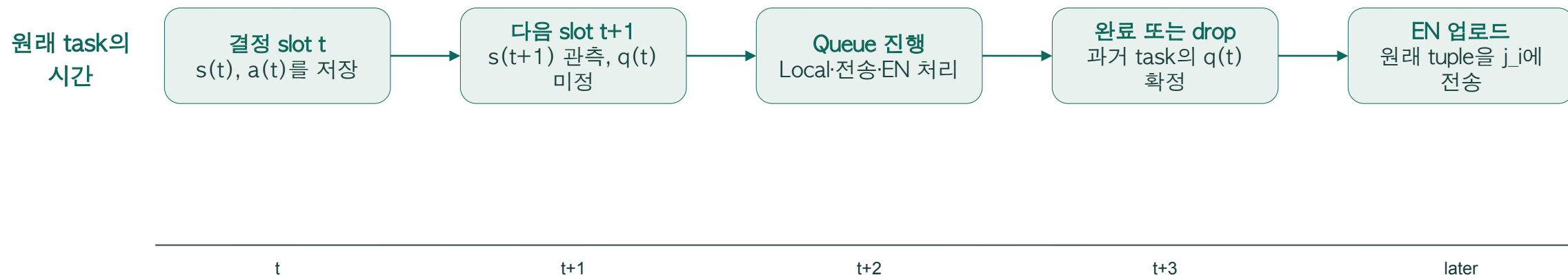
$R=10$, $\phi = 0.75$, $D=2$, $E=4$ 를 가정해 계산하되 논문 실험값과 구분한다

결과	Cost 계산	QoE 계산	값
Processed	$0.75 \times 2 + 0.25 \times 4$	$10 - 2.5$	7.5
Dropped	사용 energy $E=4$	$-E$	-4
ϕ 가 0.25라면	$0.25 \times 2 + 0.75 \times 4$	$10 - 3.5$	6.5
주의	모두 정규화 score 가정	Paper parameter가 아님	단위 해석 금지

같은 D 와 E 에서도 ϕ 가 바뀌면 action 평가가 달라지고 drop에는 completion reward가 없다

Reward가 여러 slot 뒤에 관측되는 이유

Action 시점의 task는 queue를 거친 뒤 완료 또는 drop될 때 QoE가 확정된다



Algorithm 1은 현재 끝난 과거 task의 QoE를 모아 원래 experience를 전송한다

DQN이 학습하는 값

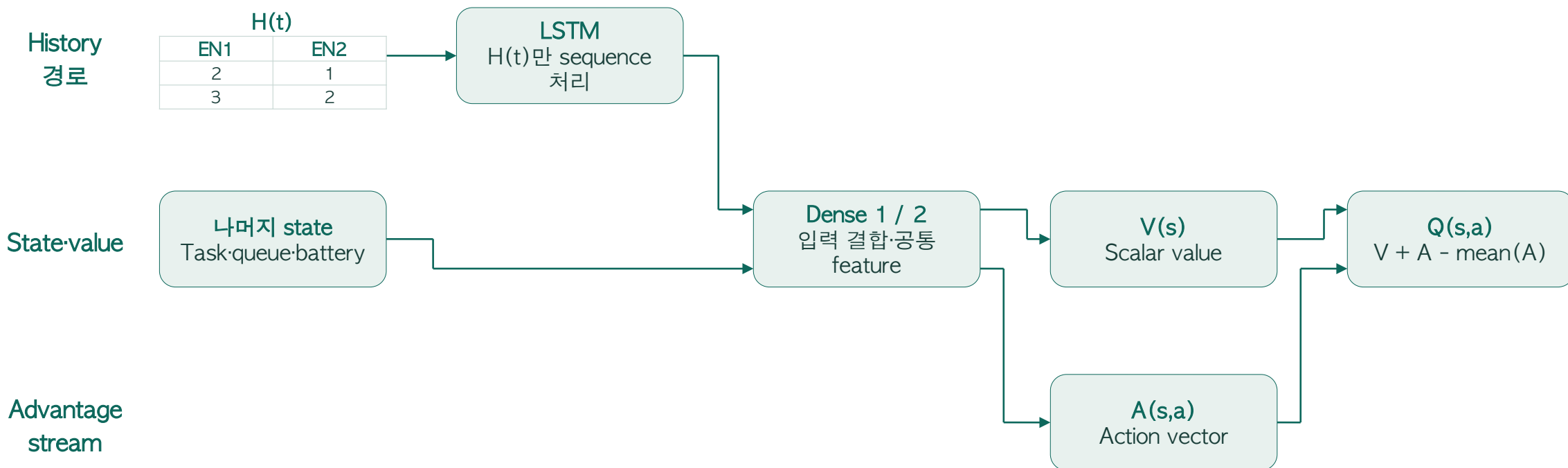
각 discrete action의 Q-value는 현재 QoE와 이후 discounted QoE를 함께 평가한다

- | | | |
|----|--------------------|---|
| 01 | State s | Task, queue, battery, workload history를 관측 |
| ↓ | | |
| 02 | Q vector | Local과 각 EN offload action의 장기 QoE를 추정 |
| ↓ | | |
| 03 | ϵ -greedy | 확률 $1-\epsilon$ 로 argmax, 확률 ϵ 로 random action |
| ↓ | | |
| 04 | Experience | 종료 뒤 QoE를 포함한 sample로 Q 오차를 학습 |

DQN은 workload를 직접 최적화하는 예측기보다 state와 action의 장기 QoE를 근사하는 함수다

D3QN feature에서 Q vector까지

LSTM과 일반 state가 만든 공통 feature를 value와 advantage branch가 공유한다



D3QN을 이루는 세 역할

LSTM, dueling, Double DQN은 같은 문제를 세 번 부르는 이름이 아니다

구성	처리 대상	QECO에서 맡는 역할
LSTM	$H(t)$ 의 EN workload sequence	과거 load 변화의 feature 학습
Dueling	공통 feature	$V(s)$ 와 $A(s,a)$ 를 나눠 Q 구성
Double DQN	다음 state의 target	Action 선택과 평가 network 분리
DQN frame	State-action value	Discounted QoE를 Q-value로 근사

D3QN은 dueling과 Double DQN을 결합하고 LSTM feature를 더한 Q-network다

Double DQN target의 선택과 평가

Evaluation network가 다음 action을 고르고 target network가 그 action의 Q-value를 계산한다

01	다음 state	Replay sample의 $s(n+1)$ 을 두 network 계산에 사용
↓		
02	Evaluation net	$\operatorname{argmax}_a Q^E(s(n+1), a)$ 로 다음 action을 선택
↓		
03	Target net	선택된 action의 Q^T 값을 평가
↓		
04	Target Q	현재 Q^E $q(n)$ 와 γ 곱한 target 값을 더함
↓		
05	Evaluation update	$Q^E(s(n), a(n))$ 와 target의 제곱 오차를 줄임

Dueling은 forward architecture이고 Double DQN은 replay sample의 target을 만드는 규칙이다

MD inference와 EN training의 소유권

MD는 parameter를 받아 action을 고르고 associated EN은 MD별 학습 상태를 유지한다

주체	보유 또는 수행	주고받는 것
MD i	현재 state 관측과 ϵ -greedy inference	UpdateRequest, experience 전송
Training EN j_i	MD i 전용 replay buffer M_i	완료한 task의 experience 수신
Evaluation net E_i	Action Q와 학습 대상 parameter	θ_i^E 를 MD에 반환
Target net T_i	Target Q 평가	주기적으로 θ_i^E 를 복사

MD는 추론하고 associated EN은 MD별 replay와 network를 학습한다

Associated EN의 replay 학습

Task가 끝난 뒤 도착한 experience가 replay sampling과 두 network update를 시작한다

01	Experience 수신	MD i 가 s, a, q, s_{next} 를 EN j_i 에 전송
↓		
02	Replay 저장	EN은 MD i 전용 buffer M_i 에 기록
↓		
03	Minibatch sample	Buffer에서 experience collection N 을 random 추출
↓		
04	Evaluation update	Double DQN target과 Q^E 의 제곱 오차를 최소화
↓		
05	Target copy	ReplaceThreshold마다 θ^E 를 θ^T 로 복사

Replay와 target network는 delayed experience를 재사용하면서 학습 target의 변화를 늦춘다

한 experience가 다음 선택을 바꾸는 과정

QECO의 인과 흐름은 parameter 요청, route 실행, delayed QoE, EN 학습, 다음 parameter 응답으로 이어진다

01	Parameter 요청	새 task를 받은 MD가 associated EN에서 θ^E 를 받음
↓		
02	Action과 실행	MD가 route와 EN target을 고르고 queue 처리를 시작
↓		
03	Delayed outcome	여러 slot 뒤 processed 또는 dropped QoE를 관측
↓		
04	EN training	Experience를 replay하고 evaluation net을 update
↓		
05	다음 선택	나중 요청에서 갱신된 θ^E 가 MD inference에 사용

현재 action의 결과는 EN replay를 거쳐 이후 task의 Q-value에 반영된다

논문에서 distributed가 뜻하는 범위

Action은 MD별 제한 관측으로 분산되지만 학습은 associated EN에서 MD별로 수행된다

질문	QECO의 답	포함하지 않는 구조
누가 action을 고르는가	각 MD가 자신의 state로 선택	중앙 controller의 joint action
다른 MD 정보가 필요한가	Task model과 action은 불필요	모든 MD의 policy 공유
누가 network를 학습하는가	Associated EN이 MD별 학습	MD 내부 end-to-end training
EN load는 어떻게 아는가	Broadcast history $H(t)$ 를 관측	미래 workload 완전 관측

QECO의 분산성은 제한된 관측을 이용한 MD별 action 선택에 있다

Simulation과 training 조건

실험은 실제 dataset 없이 고정된 MEC simulation에서 1,000 episode를 학습한다

범주	설정	해석
System 규모	기본 50 MD, 5 EN	부하와 MD 수를 별도 변화
Episode	1,000회, 각 100 slot	Slot 길이 $\tau = 0.1$ s
Learning	Batch 16, rate 0.001, $\gamma = 0.9$	RMSProp 사용
Exploration	ϵ 가 1에서 0.01로 감소	초기 random 선택에서 점차 greedy
Data	실제 dataset 없음	Simulation에서 experience 수집

결과는 이 simulation과 parameter 범위에서 얻은 값이다

주요 MEC parameter

Table II는 계산 성능, 전송률, task 분포, deadline, power 값을 고정한다

Parameter	값	모델에서의 역할
Compute capacity	MD 2.6 GHz, EN 42.8 GHz	Local과 edge 처리 속도
Transmission	14 Mbps, MD power 2.3 W	Offload delay와 energy
Task	1.0- 7.0 Mbits, arrival 150 task/s	기본 workload
CPU density	$0.197, 0.297, 0.397 \times 10^3$ cycle/bit	Task 계산량
Deadline과 battery	10 slot, 25%, 50%, 75%	1 s 제한과 QoE weight

다른 MEC 환경에 적용할 때 Table II의 task와 battery 조건을 함께 확인해야 한다

Task 성능 비교 대상

LC, FO, RD는 단순 정책이고 PGOA와 DCDRL은 기존 offloading 방법이다

Method	Action 방식	비교 의미
LC	모든 task를 MD에서 계산	Local-only 기준
FO	모든 task offload, EN은 random	Offload-only 기준
RD	Local 또는 offload와 target을 random	학습 없는 선택
PGOA	Delay-sensitive distributed optimization	전략적 MD-EN 상호작용
DCDRL	Actor-critic 기반 distributed DRL	Queuing MEC의 기존 DRL

DQN과 DDQN은 Fig. 7의 convergence 비교이며 이 task 성능 baseline 목록과 목적이 다르다

완료 task와 energy 결과

비교 비율은 workload 조건과 comparator를 함께 붙여야 정확하다

조건	Metric	QECO 결과	Comparator
Arrival rate 250	Completed task	+72.1%, +46.8%, +28.6%	RD, PGOA, DCDRL 순서
110 MD	Completed task	최소 +22.8%	다른 방법 전체
Arrival rate 450	Total energy	-18.6%, -15.5%, -13.9%	RD, PGOA, DCDRL 순서
110 MD	Total energy	-27.4%, -16.7%, -23.5%	RD, PGOA, DCDRL 순서
Energy 예외	LC와 FO 비교	QECO가 더 많이 소비	LC/FO 자원 제한

QECO는 세 비교법보다 개선됐지만 LC와 FO보다 energy가 낮지는 않았다

Delay와 QoE 결과의 조건부 해석

High-load에서 더 많은 task를 완료하면 평균 delay가 오히려 높아질 수 있다

조건	Metric	QECO 결과	해석
Arrival rate 200	Average delay	최소 12.4% 낮음	다른 방법 대비
Arrival rate >350	Average delay	일부 방법보다 높을 수 있음	타 방법의 drop 증가
70 MD	Average QoE 0.57	+70.3%, +44.7%	PGOA, DCDRL 순서
110 MD	Average QoE	최소 +24.8%	다른 방법 대비
Arrival 350-450	Average QoE	+42.6%, +28.5%, +22.6%	RD, PGOA, DCDRL 순서

완료 수, 평균 delay, energy, QoE를 함께 읽어야 drop이 만든 비교 착시를 피할 수 있다

Network 구성별 empirical convergence

Fig. 7은 평균 QoE가 안정되는 iteration을 비교하지만 수학적 수렴 보장은 제공하지 않는다

Network	안정화 시점	포함 구성	읽는 법
Vanilla DQN	약 650 iteration	기본 Q-network	Simulation curve
DDQN	약 550 iteration	Double target rule	Simulation curve
QECO	약 450 iteration	LSTM과 D3QN	Simulation curve
Formal guarantee	제시되지 않음	Function approximation DRL	논문이 open problem으로 명시

약 450회는 해당 simulation의 안정화 시점이며 수렴 증명이 아니다

결과를 옮길 때 남는 한계

Simulation 성과와 실제 mobile-edge deployment 사이에는 측정되지 않은 구간이 남아 있다

한계	원문 근거	해석 영향
실제 dataset 없음	Simulation으로 experience 생성	현장 traffic 일반화 미검증
QoE scaling 미기재	D와 E normalization, R 수치 불명확	절대 score 이식에 주의
미래 EN load 불확실	$H(t)$ 는 과거 active queue count	LSTM 예측 정확도 별도 실험 없음
Task dependency 제외	Call graph는 future work	독립 task 모델 범위
Convergence 보장 없음	논문이 open problem으로 명시	Fig. 7은 empirical evidence

실제 배포의 비용과 안정성은 이 queue simulation 밖에서 검증해야 한다

QECO의 정확한 기술 경계

논문 이름, 실행 위치, 학습 구조를 분리하면 후속 연구와의 혼동을 줄일 수 있다

경계	이 논문의 QECO	구분해야 할 대상
논문 정체성	2025 TNSE의 QoE offloading	별도 QECO-Adapt 연구
MD 역할	State 관측과 action inference	MD 내부 전체 training
EN 역할	MD별 replay와 D3QN training	Task offload target과 동일시
학습 방식	EN-assisted per-MD Double DQN	Federated learning과 actor-critic
근거 범위	Queue simulation의 conditioned result	실제 on-device 성능 보장

QECO $\neq$ QECO-Adapt이며 MD inference $\neq$ MD training이다

QECO의 핵심 흐름

한 task의 route 선택에서 시작해 delayed QoE와 EN-assisted learning으로 돌아오는 구조다

- MD는 local queue 또는 특정 EN으로 가는 discrete action을 고른다
- State는 MD backlog와 battery, EN별 내 queue와 workload history를 포함한다
- Task가 끝나야 QoE가 확정되고 experience가 associated EN으로 간다
- EN은 MD별 replay와 두 network를 학습해 다음 parameter를 제공한다
- Simulation은 completion, delay, energy, QoE의 tradeoff를 조건별로 보여 준다

QECO는 MD별 선택과 EN-assisted D3QN training을 결합한 queue 정책이다

공개 출처

논문 기록과 공개 manuscript, 저자가 논문에 인용한 code repository를 구분한다

출처	용도	식별 정보
Publisher DOI	정식 논문 식별자	10.1109/TNSE.2025.3556809
IEEE Xplore	Journal article record	Document 10946841
arXiv v3	공개 manuscript record	arXiv:2311.02525v3
Author code	재현성 확인용 repository	github.com/lmanRHT/QECO

알고리즘과 수치는 journal paper, code는 구현 자료로 구분한다